

# Add 算子测试报告

本文以 **Add 算子** 为例，记录测试环境、测试设计、执行过程、结果验证、覆盖率分析以及后续优化方向，作为一个可复用的算子测试报告模板。

## 1. 测试目标

本次测试的目标是针对 **Add 算子** 编写并扩展测试用例，在 CPU 模拟器环境中执行，并结合覆盖率工具分析测试效果。通过增加不同数据类型、不同 shape、不同 API 变体的测试场景，验证算子计算结果的正确性，并尽可能提升关键源码文件的代码覆盖率。

本次测试重点关注以下几个方面：

- **功能正确性**：验证算子及其相关 API 在不同输入场景下输出结果是否正确；
- **场景覆盖性**：覆盖更多数据类型、广播场景和 API 路径；
- **覆盖率提升**：通过 gcov 分析关键文件的代码覆盖率变化；
- **可复用性**：形成可持续扩展的测试模板，为后续其他算子的测试提供参考。

## 2. 测试环境

### 2.1 软件环境

- Docker 镜像：[yeren666/cann-ops-test:v1.0](#)
- 操作系统：Linux x86\_64（Docker 容器内）
- 工具链：CANN Toolkit
- 覆盖率工具：gcov
- 编译方式：[build.sh --pkg --cov](#)
- 运行方式：CPU SiAddator（模拟器模式）

### 2.2 环境准备命令

```
docker pull yeren666/cann-ops-test:v1.0
docker run -it yeren666/cann-ops-test:v1.0

cd /home/workspace/ops-math
```

如镜像离线导入：

```
docker load -i cann-ops-test-v1.0.tar.gz
docker run -it --name add-test cann-ops-test:v1.0
```

## 3. 被测对象说明

### 3.1 算子名称

- 算子名称: Add
- 功能描述: 对两个输入 Tensor 做逐元素加法运算

### 3.2 涉及的关键源码文件

本次测试重点关注以下四个文件的覆盖率:

- `math/add/op_api/aclnn_add.cpp`
- `math/add/op_api/aclnn_add_v3.cpp`
- `math/add/op_api/add.cpp`
- `math/add/op_host/arch35/add_tiling_arch35.cpp`

### 3.3 `test_aclnn_inplace_add.cpp` 在模拟器上崩溃。

官方 `test_aclnn_inplace_add.cpp` 在 CPU 模拟器环境下运行时会触发内存错误 (`malloc(): unaligned tcache chunk detected`) 导致程序崩溃。

**解决方法:** 将 `math/add/examples/test_aclnn_inplace_add.cpp` 替换为以下占位脚本:

```
#include <stdio>

int main()
{
    std::printf("skip standalone test_aclnn_inplace_add, covered by
test_aclnn_add.cpp\n");
    return 0;
}
```

## 4. 测试策略

### 4.1 测试维度

本次测试从以下几个维度设计测试用例:

#### (1) 数据类型维度

不同 dtype 会触发不同的调度和 tiling 分支, 例如:

- `float32`
- `int32`
- `float16`
- `bool`
- 混合类型 (如 `float16 * float32`)

#### (2) 输入 shape 维度

不仅测试相同 shape, 也考虑广播场景:

- 同 shape:  $\{4, 2\} * \{4, 2\}$
- 广播 shape:  $\{2, 3\} * \{1, 3\}$

### (3) API 变体维度

覆盖 Add 家族的不同 API 路径:

- `aclnnAdd`
- `aclnnAddds`
- `aclnnInplaceAdd`
- `aclnnInplaceAddds`
- `aclnnAddV3`
- `aclnnInplaceAddV3`

### (4) 边界值维度

增加零值、负值、特殊情况输入:

- 含负数
- 含零
- 空指针或非法输入 (异常分支)

---

## 5. 测试用例设计示例

本次测试针对 Add 算子的 `aclnnAddGetWorkspaceSize` 接口进行用例扩展, 重点覆盖标准 Tensor-Tensor 加法、广播场景、类型提升场景以及高维边界场景。相比只测试单一同 shape、同 dtype 的基础用例, 本组用例能够触发更多 shape 推导、dtype 处理和 tiling 相关分支, 从而提升代码覆盖率。

---

### 5.1 用例一: float32 基础 Tensor-Tensor 加法

**目的:** 验证 Add 算子在最基础的同 shape、同 dtype 场景下能够正常完成 workspace 计算, 并覆盖标准 `Tensor + Tensor` 路径。

- 输入:
  - `t1 shape = {2, 4}`
  - `t2 shape = {2, 4}`
  - `t0ut shape = {2, 4}`
  - `dtype = float32`
  - `alpha = 1.0`
- API:
  - `aclnnAddGetWorkspaceSize`
- 计算语义:
  - `out = t1 + alpha * t2`
- 预期结果:
  - 接口返回成功;
  - 能够正确生成 `aclOpExecutor`;
  - `workspaceSize` 能够正常返回;

- 不发生异常或崩溃。

---

## 5.2 用例二：广播 shape 加法

**目的：**覆盖广播处理逻辑，验证不同 shape 输入在满足广播规则时能正确完成 shape 推导与 workspace 计算。

- 输入：
  - `t1` shape = {2,1}
  - `t2` shape = {1,4}
  - `tOut` shape = {2,4}
  - `dtype` = float32
  - `alpha` = 1.0
- API：
  - `aclnnAddGetWorkspaceSize`
- 广播关系：
  - {2,1} + {1,4} -> {2,4}
- 计算语义：
  - `out = t1 + alpha * t2`
- 预期结果：
  - 接口返回成功；
  - 广播输出 shape {2,4} 被正确接受；
  - 触发广播相关 shape 处理路径；
  - 不发生 shape 不匹配错误。

---

## 5.3 用例三：类型提升测试 (float16 + int32)

**目的：**覆盖不同 dtype 输入下的类型提升逻辑，验证 float16 与 int32 混合输入时能够正确完成类型推导与接口调度。

- 输入：
  - `tF16` shape = {4}, dtype = float16
  - `tI32` shape = {4}, dtype = int32
  - `tOut` shape = {4}, dtype = float32
  - `alpha` = 1.0
- API：
  - `aclnnAddGetWorkspaceSize`
- 计算语义：
  - `out = tF16 + alpha * tI32`
- 预期结果：
  - 接口返回成功；
  - 混合类型输入能够被正确识别；
  - 输出 Tensor 使用较高精度 float32；
  - 覆盖 dtype promotion 相关分支。

---

## 5.4 用例四：8 维边界 shape 测试

**目的：**覆盖高维 Tensor 的边界场景，验证 Add 算子在 8 维输入下能正常完成 shape 分析与 workspace 计算。

- 输入：
  - `t1 shape = {1,1,1,1,1,1,1,8}`
  - `tOut shape = {1,1,1,1,1,1,1,8}`
  - `dtype = float32`
  - `alpha = 1.0`
- API：
  - `acInnAddGetWorkspaceSize`
- 计算语义：
  - `out = t1 + alpha * t1`
- 预期结果：
  - 接口返回成功；
  - 8 维 shape 能够被正确处理；
  - 覆盖高维 Tensor 边界处理逻辑；
  - 不发生维度解析异常。

---

## 5.5 可扩展用例（后续优化方向）

- 增加真实 device 数据并调用 `acInnAdd` 第二段接口，验证实际计算结果；
- 增加 `int32 + int32`、`float16 + float16`、`bool + bool` 等更多 dtype 组合；
- 增加负数、零值、极大值、极小值等边界数据测试；
- 增加 `alpha != 1.0` 的测试，例如 `alpha = 0.5`、`alpha = -1.0`；
- 增加 `acInnInplaceAdd` 的测试，覆盖原地加法路径；
- 增加 `acInnInplaceAdd`s 的测试，覆盖 Tensor 与 Scalar 原地加法路径；
- 增加异常输入测试，例如空指针、非法 shape、不兼容广播 shape、输出 dtype 不匹配等场景。

---

## 6. 测试代码实现说明

本次测试基于 Add 算子的 ACLNN 接口进行扩展，主要调用的是第一段接口 `acInnAddGetWorkspaceSize`。该接口用于根据输入 Tensor、`alpha` 标量和输出 Tensor 信息完成参数检查、shape 推导、dtype 处理、tiling 准备以及 executor 创建。

本次测试代码主要新增和覆盖了以下内容：

1. 构造基础同 shape 的 `float32 + float32` 用例，覆盖标准 `Tensor + Tensor` 路径；
2. 构造 `{2,1} + {1,4} -> {2,4}` 的广播用例，覆盖广播 shape 推导路径；
3. 构造 `float16 + int32 -> float32` 的混合类型用例，覆盖 dtype promotion 类型提升逻辑；
4. 构造 8 维 Tensor 用例，覆盖高维 shape 边界处理路径；
5. 每个用例均创建输入 Tensor、输出 Tensor 和 `alpha` Scalar，并调用 `acInnAddGetWorkspaceSize` 验证接口是否能成功返回；
6. 每个用例结束后主动释放 `acIScalar` 和 `acITensor`，避免资源泄漏。

## 7. 测试执行过程

### 7.1 编译算子

```
bash build.sh --pkg --soc=ascend950 --ops=add --vendor_name=custom --cov
```

7.2 安装算子包

```
./build_out/cann-ops-math-custom_linux-x86_64.run --quiet
```

7.3 运行测试示例

```
bash build.sh --run_example add eager cust \  
--vendor_name=custom --simulator --soc=ascend950 --cov
```

8. 测试结果

8.1 功能测试结果

运行后输出如下：

```
run test_aclnn_add, execute samples success
```

说明当前设计的测试用例全部通过，Add 算子在这些场景下输出结果正确。

9. 覆盖率分析

9.1 扩展测试后的覆盖率

新增测试用例后再次统计覆盖率：

| 文件                    | 行覆盖率   | 分支覆盖率  | 提升说明         |
|-----------------------|--------|--------|--------------|
| add.cpp               | 60.00% | 31.06% | 测试了多种精度      |
| aclnn_add_v3.cpp      | 71.43% | 58.30% | 测试了V3特性，校验分支 |
| aclnn_add.cpp         | 62.72% | 50.38% | 非连续性调用多个函数   |
| add_tiling_arch35.cpp | 82.22% | 61.25% | 拆分多核，选择切分策略  |

9.2 覆盖率不足原因分析

当前仍有部分分支未覆盖，主要原因包括：

- 1. 异常输入与错误处理路径未覆盖；

## 10. 后续优化方向

为了进一步提升覆盖率，可继续从以下方向优化：

### 10.1 增加更多数据类型

尝试以下类型组合：

- `int32 * int32`
- `float16 * float32`
- `int8 * int8`
- `double * double`

### 10.2 增加广播测试

例如：

- `shape1 = {2,3}`
- `shape2 = {1,3}`

用于触发广播路径。

### 10.3 增加 Inplace API 测试

覆盖以下函数：

- `aclnnInplaceAddGetWorkspaceSize`
- `aclnnInplaceAdd`
- `aclnnInplaceAddsGetWorkspaceSize`
- `aclnnInplaceAdds`

### 10.4 增加异常路径测试

例如：

```
aclnnAddGetWorkspaceSize(nullptr, x2T, outT, &wsSize, &executor);
```

验证返回错误码而非程序崩溃。

---

## 11. 结论

本次测试基于官方示例，对 Add 算子的测试用例进行了扩展与完善。相比原始示例，新增了结果验证机制，并增加了不同输入场景和 API 变体的测试，从而有效提升了关键文件的覆盖率。实验结果表明，合理设计测试维度（dtype、shape、API、边界值）能够明显提高算子测试质量和代码覆盖率。

当前测试已完成基础模板搭建，后续可继续扩展更多数据类型和异常场景，以进一步提升覆盖率和测试完备性。本报告也可作为其他算子测试工作的通用模板使用。

---

## 12. 附录

### 12.1 覆盖率查看命令

```
gcov -b $(find build -name "aclnn_add.cpp.gcda" | head -1) 2>&1 \  
| grep -A2 "File.*aclnn_add.cpp" | head -3
```

### 12.2 查看具体未覆盖代码行

```
gcov -b $(find build -name "aclnn_add.cpp.gcda" | head -1)  
cat aclnn_add.cpp.gcov | head -50
```

### 12.3 常见问题

#### 1) 退出容器后数据是否丢失？

不会，可以使用以下命令重新进入：

```
docker start -i add-test
```

#### 2) 覆盖率没有变化怎么办？

确认是否完整执行了：

**编译 → 安装 → 运行**

---